BCIS Technical Exercise

Interview Technical Assessment

Time allocation	Approximately 2 hours in total (see individual tasks for guidance)
Format	Your choice of language and tooling unless stated otherwise
Submission	Share your code and any output files (e.g. GitHub link, zip, or notebook) ahead of the interview

We are interested in how you approach problems, not just the final answer. Clear, readable code with brief explanations of your thinking is more valuable than a terse solution that produces the right number.

Background

BCIS collects data about construction projects and buildings from across the UK. The two tasks below reflect different aspects of the data and systems work we do: one tests algorithmic reasoning, the other replicates part of our data extraction and analysis pipeline.

Task A — Survey Lifecycle Simulation

Suggested time: 30–60 minutes | Language: your choice

Context

Each completed survey in the BCIS system may generate follow-up surveys after a certain number of days — for example, re-inspection or verification work. Survey volume therefore grows over time in a way that the analytics team needs to forecast.

Each survey behaves as a simple lifecycle timer:

- Every survey has a **countdown timer** representing days until its next follow-up is generated.
- Each day, all timers decrease by **1**.
- When a timer reaches **0**:
  - The survey generates **one new follow-up survey**.
  - The original survey's timer resets to **7** days.
  - The new follow-up survey starts with a timer of **9** days.

Input

Your input is a single line of comma-separated integers, each representing the current countdown timer of a survey already active in the system. A small example:

Str

```
2,4,3,1,3
```

EXAMPLE

Your full input is provided in `input.txt`.

- TEST ALGORITHM WITH RESULT HIGHLIGHT IN RED

Part 1

Simulate **80 days** of survey activity.

Using the example above, after 18 days there are **21** surveys in the system, and after 80 days there are **2,773**.

How many surveys are active after **80 days** using the input in `input.txt`? TXT FILE

Part 2

The BCIS portfolio is considerably larger than the initial batch suggests.

How many surveys are active after **256 days**? CIRCULAR BUFFER APPROACH!

> ⚠️ The answer for Part 2 is a large number. Make sure your solution can handle it.

Deliverable

Working code that reads from `input.txt` and prints the answer for each part, with a brief note explaining your approach — particularly for Part 2. USE OF CIRCULAR BUFFER WILL LIMIT THE AMOUNT OF DATA STORED IN-MEMORY

Task B — EPC Data Analysis

Suggested time: approximately 1 hour | Language: Python

Context

A key use of data within BCIS is understanding the characteristics and variation of UK housing stock. This task replicates a small part of our data extraction and analysis pipeline, working with two open government datasets that we use regularly.

You will be joining postcode reference data from the ONS Postcode Directory against domestic EPC (Energy Performance Certificate) records to derive a set of summary statistics aggregated by postcode outcode.

A **postcode outcode** is the first part of a full postcode — for example, the outcode of `SW1A 1AA` is `SW1A`. Each outcode typically covers thousands of individual properties across several streets or a neighbourhood.

Data Provided

The following two files are provided alongside this brief. They are subsets of larger public datasets (see Stretch Goals for links to the full versions).

File	Description
`postcodes_100k.csv`	100,000 randomly sampled active postcodes from the ONS Postcode Directory (November 2025 edition), covering England and Wales only. Includes the postcode, local authority district code, region code, and WGS84 latitude/longitude.
`epc_subset.csv`	All EPC records from the 2024 England and Wales domestic dataset whose `POSTCODE` falls within the 100,000 postcode sample. Contains approximately 68,000 records across 2,200+ outcodes.

Schema Reference

`postcodes_100k.csv` — derived from the ONS Postcode Directory.

Column	Type	Description
`pcds`	string	Full postcode in standard format (e.g. `SW1A 1AA`)
`lad25cd`	string	2025 Local Authority District code
`rgn25cd`	string	2025 Region code (England regions + Wales)
`lat`	float	WGS84 latitude
`long`	float	WGS84 longitude

> 💡 **Note on postcode format:** The `pcds` field uses the standard spaced format (e.g. `SW1A 1AA`). The `POSTCODE` field in the EPC file uses the same format, so no normalisation is required for a direct join.

`epc_subset.csv` — derived from the 2024 England and Wales Domestic EPC dataset published at epc.opendatacommunities.org.

Column	Type	Description
`LMK_KEY`	string	Unique identifier for the EPC lodgement
`POSTCODE`	string	Full postcode of the property
`CURRENT_ENERGY_RATING`	string	EPC energy efficiency band (A–G)
`PROPERTY_TYPE`	string	House, Flat, Bungalow, Maisonette, Park home
`BUILT_FORM`	string	Detached, Semi-Detached, Terraced, End-Terrace, Enclosed End-Terrace, Enclosed Mid-Terrace
`INSPECTION_DATE`	date	Date of the EPC inspection (YYYY-MM-DD)
`TOTAL_FLOOR_AREA`	float	Total floor area in square metres
`NUMBER_HABITABLE_ROOMS`	integer	Count of habitable rooms (excludes bathrooms, WCs, halls)
`CONSTRUCTION_AGE_BAND`	string	Broad construction era band (e.g. 1967-1975)
`TENURE`	string	Owner-occupied, rental (social), rental (private), unknown
`TRANSACTION_TYPE`	string	Reason for the EPC (e.g. marketed sale, rental, new dwelling)

Part 1 — Postcode outcode coverage

- TRIM POSTCODE TO PRODUCE OUTCODE

Using only postcodes that appear in `postcodes_100k.csv`, find the **10 postcode outcodes** with the highest number of EPC records in `epc_subset.csv`.

ASCENDING = FALSE

Output a table showing the outcode and its EPC record count, sorted descending.

Notes:

- An outcode is the first part of a full postcode: the outcode of `SW1A 1AA` is `SW1A`.
- Only include EPC records whose postcode exists in `postcodes_100k.csv`. Records with postcodes not in the 100k list should be excluded.

Part 2 — Median habitable rooms by outcode

. MEDIAN FUNCTION

Using the same filtered EPC records as Part 1, calculate the **median number of habitable rooms** for each postcode outcode.

Output a table showing the **10 outcodes with the highest median habitable rooms**, sorted descending. Include the outcode, median habitable rooms, and the number of EPC records used to compute the median.

Notes:

- Exclude EPC records where `NUMBER_HABITABLE_ROOMS` is null or zero.
- Exclude outcodes with fewer than **10 valid records** — too small a sample to be meaningful.

Deliverable

— U.I. CAN BE INTRODUCED – TO SEE SELECT VIEW

A Python script or notebook that can be run locally from the provided files, producing the output tables for both parts. Brief inline comments explaining any non-obvious steps are appreciated.

- OPTION LIST.

Stretch Goals

These are genuinely optional. A well-executed solution to the core tasks is more valuable than a rushed stretch.

Stretch A — Parliamentary Constituency analysis

DOWNLOADED A COPY

A GeoJSON boundary file for Westminster Parliamentary Constituencies (July 2024) is provided alongside this brief. Using the `lat` and `long` columns in `postcodes_100k.csv`, perform a **point-in-polygon spatial join** to assign each postcode to a constituency, then repeat the median habitable rooms analysis at constituency level instead of outcode.

> 💡 **Hints:**
>
> - `geopandas` is the natural fit here — `gpd.read_file()` loads the GeoJSON and `gpd.sjoin()` (with `predicate="within"`) handles the join.
> - Construct a `GeoDataFrame` from the postcode coordinates first: `gpd.GeoDataFrame(df, geometry=gpd.points_from_xy(df.long, df.lat), crs="EPSG:4326")`.
> - Ensure both GeoDataFrames share the same CRS before joining — the GeoJSON uses WGS84 (`EPSG:4326`).

- READ CSV + JSON VIA GEOPANDAS
- SORT OUT CRS FORMAT
- JOIN POLYGONS FROM GEOJSON TO CENTROIDS POINTS WITHIN THE POSTCODE CSV

Stretch B — Scale to the full national datasets

Rewrite your pipeline to handle the full public data. The full datasets are significantly larger (the ONSPD has ~2.7 million rows; the full EPC dataset ~28.9 million rows), so consider how you would approach this efficiently.

- **Full ONS Postcode Directory (November 2025):** https://geoportal.statistics.gov.uk/datasets/a2f8c9c5778a452bbf640d98c166657c/about
- **Full England and Wales Domestic EPCs (all years):** https://epc.opendatacommunities.org/ (register for a free account, then download "All domestic EPCs")

Stretch C — Visualisation

Produce a visualisation of your results. A choropleth map, distribution chart, or ranked bar plot would all be appropriate. If you use the GeoJSON boundary file or the postcode coordinates, briefly comment on how you approached the spatial element.

STREAMLIT - DASHBOARD TO DISPLAY GEOSPATIAL MAP + OTHER ANALYTICS.

Evaluation Criteria

Across both tasks we are primarily interested in:

- **Correctness** — the right answer via sound reasoning
- **Code clarity and structure** — readable, appropriately commented, sensible naming
- **Approach to scale** — particularly in Task A Part 2 and Task B Stretch B: can you articulate why a naive solution fails and how to fix it?
- **Awareness of data quality** — nulls, edge cases, format inconsistencies (Task B)
- **Use of appropriate tooling** — there is no requirement to use any specific library or framework; a clean, idiomatic solution in your preferred stack is the goal

We will discuss your submission during the interview, so be prepared to walk through your reasoning.

GEOJSON - GEOMETRY KEY CONTAINED NULL VALUES
> GEOJSON WAS DOWNLOADED FROM WEBSITE MENTIONED

Python packages:
- PANDAS, DUCKDB, GEOPANDAS